Patrones de comportamiento

Hoja de ejercicios (Alumno)

Patrón *Observer*

Escenario

Una aplicación de control de misión espacial monitoriza la telemetría de una nave. La telemetría incluye parámetros relacionados con la navegación como la posición, la orientación, la velocidad y la velocidad de rotación.

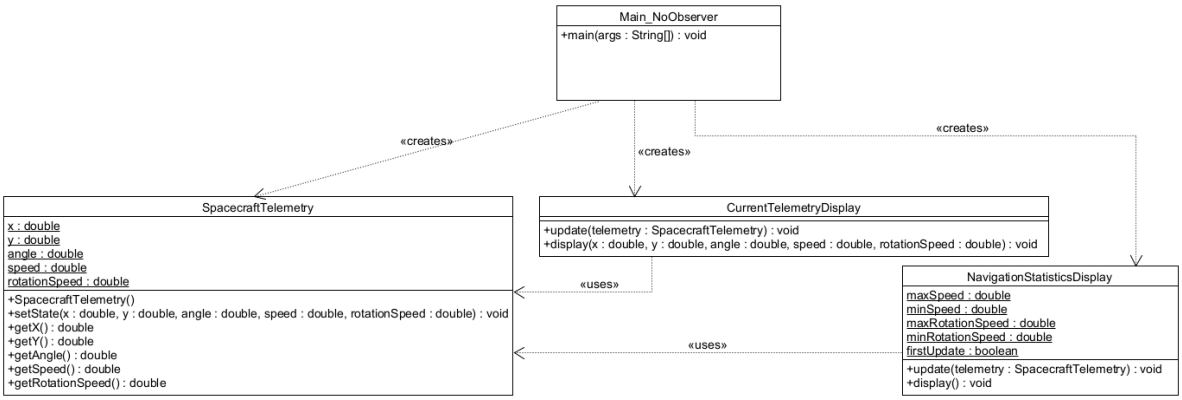
Diferentes componentes visuales deben presentar esta información en tiempo real. Por ejemplo:

- Un visor que muestra los valores actuales de la telemetría.
- Un visor que calcula estadísticas de navegación como velocidades mínimas y máximas.

El sistema debería estar preparado para permitir añadir nuevas *vistas* de la información sin necesidad de modificar la lógica principal de la aplicación.

Se proporciona a los estudiantes un proyecto inicial a través del campus virtual.
Este proyecto contiene una solución funcional que **no utiliza ningún patrón de diseño**.

El siguiente diagrama UML representa la estructura del sistema entregado.



Limitaciones observadas

En la implementación proporcionada:

- La aplicación principal crea y gestiona directamente el objeto de telemetría y todos los componentes visuales.
- Cada vez que cambia la telemetría, la aplicación principal debe invocar manualmente las operaciones de actualización de cada visor.
- No existe ningún mecanismo que vincule la información de telemetría y sus visualizaciones, por lo que éstas no saben cuándo cambia la información de telemetría.

- Añadir un nuevo visor implica modificar la lógica de la aplicación principal.

Esto produce un acoplamiento fuerte entre la gestión de la telemetría y los componentes encargados de su visualización.

Tarea

Se debe rediseñar el sistema aplicando el **patrón Observer**.
La solución deberá introducir los siguientes elementos.

Interfaz Subject

Crear una interfaz denominada `Subject` con los métodos:

```
registerObserver(o : Observer) : void
removeObserver(o : Observer) : void
notifyObservers() : void
```

Interfaz Observer

Crear una interfaz denominada `Observer` con el método:

```
update() : void
```

Sujeto concreto

La clase de telemetría debe pasar a denominarse `SpacecraftTelemetry` e implementar la interfaz `Subject`, manteniendo la colección:

```
observers : List<Observer>
```

Debe proporcionar un método:

```
setState(x : double, y : double, angle : double,
        speed : double, rotationSpeed : double) : void
```

que actualice los datos de telemetría y notifique a todos los observadores registrados.

Observadores concretos

Las clases de visualización deben adaptarse para implementar la interfaz Observer:

```
CurrentTelemetryDisplay
NavigationStatisticsDisplay
```

Cada observador deberá:

- almacenar una referencia al objeto de telemetría,
- suscribirse automáticamente a él en el constructor,
- implementar el método update() recuperando los valores actuales.

Lógica de la aplicación principal

La aplicación principal deberá limitarse a:

- crear el objeto de telemetría,
- crear los observadores pasándoles la referencia a la telemetría,
- modificar el estado mediante llamadas a setState(...).

No deberán existir llamadas explícitas a métodos de actualización de los visores.